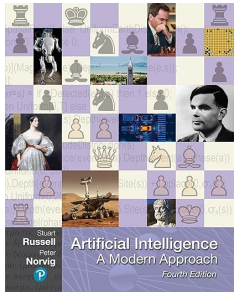
Lesson 06.2: Using Bayesian Networks



Instructor: Dr. GP Saggese, gsaggese@umd.edu

References:

- AIMA (Artificial Intelligence: a Modern Approach)
  - Chap 12, Quantifying uncertainty
  - Chap 13: Probabilistic reasoning
  - Chap 14: Probabilistic reasoning over time

- ***Semantics of Bayesian Networks***
- Constructing a Bayesian Network
- Exact Inference in Bayesian Networks
- Approximate Inference in Bayesian Networks

Bayesian Networks: Semantics

- There are **two equivalent semantic interpretations** of a Bayesian Network

  1. **Joint Distribution View**
     - The network encodes the *joint probability distribution* over all variables
     - Computed as the product of local conditional probabilities:

$$P(X_1, ..., X_n) = \prod_{i=1}^{n} P(X_i \mid \text{Parents}(X_i))$$

     - Useful for constructing models and understanding overall behavior
  2. **Conditional Independence View**
     - The structure encodes *conditional independency* between variables
     - Useful for inference and reasoning
     - A variable is conditionally independent of its non-descendants given its parents

Chain Rule for a Joint Distribution

- A **joint distribution** can always be expressed using the chain rule for any:
  - Subset of its RVs
  - Ordering of the RVs

1. You **express one variable** conditionally to the remaining ones

$$\Pr(x_1, ..., x_{n-1}, x_n) = \Pr(x_n | x_{n-1}, ..., x_1) \Pr(x_{n-1}, ..., x_1)$$

2. Apply the same formula **recursively**, until you get an unconditional probability

$$\begin{aligned}
&\Pr(x_1, x_2, ..., x_{n-2}, x_{n-1}, x_n) \\
&= \Pr(x_n | x_{n-1}, ..., x_1) \Pr(x_{n-1}, ..., x_1) \\
&= \Pr(x_n | x_{n-1}, ..., x_1) \Pr(x_{n-1} | x_{n-2}, ..., x_1) \Pr(x_{n-2}, ..., x_1) \\
&... \\
&= \Pr(x_n | x_{n-1}, ..., x_1) \Pr(x_{n-1} | x_{n-2}, ..., x_1) \Pr(x_{n-2} | x_{n-3}, ..., x_1) ... \Pr(x_2 | x_1) \Pr(x_1) \\
&= \prod_{i=1}^{n} \Pr(x_i | x_{i-1}, ..., x_1)
\end{aligned}$$

Statement Probability from Bayesian Network

- The **full joint distribution** represents the probability of an assignment to each variable $X_i = x_i$:

$$\Pr(x_1, ..., x_n) \triangleq \Pr(X_1 = x_1 \wedge ... \wedge X_n = x_n)$$

- To **evaluate a Bayesian network**
  - Sort the nodes in topological order
    - There are several orderings consistent with the directed graph structure
  - Use the chain rule with the topological ordering:

$$\Pr(X_1, ..., X_n) = \prod_{i=1}^{n} \Pr(X_i | X_{i-1}, ..., X_1)$$

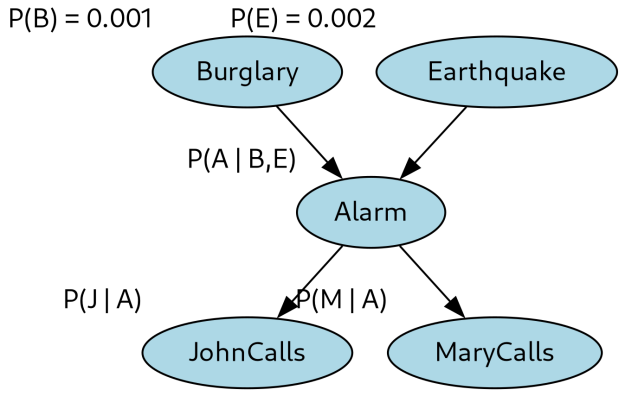
  - Since the probability of each node is conditionally independent of all its predecessors given its parents

$$\Pr(X_i | X_{i-1}, ..., X_1) = \Pr(X_i | Parents(X_i))$$

  - Express the joint probability in terms of the Conditional Probability Tables (CPTs):

Statement Probability From Bayes Nets: Example

- Given Pearl LA example, you want to compute the probability that:
  - The alarm has sounded: *Alarm*
  - Neither a burglary nor an earthquake has occurred: $\neg Burglary \wedge \neg Earthquake$
  - Both John and Mary call: *JohnCalls*, *MaryCalls*



- **Solution**
  - Compute the probability as a product of conditional probabilities from the Bayesian Network

$$
\begin{aligned}
&\Pr(JohnCalls, MaryCalls, Alarm, \neg Burglary, \neg Earthquake) \\
&= \Pr(JohnCalls|Alarm)\cdot \\
&\qquad \Pr(MaryCalls|Alarm)\cdot \\
&\qquad \Pr(Alarm|\neg Burglary \wedge \neg Earthquake)\cdot \\
&\qquad \Pr(\neg Burglary) \cdot \Pr(\neg Earthquake)
\end{aligned}
$$

- Semantics of Bayesian Networks
- ***Constructing a Bayesian Network***
- Exact Inference in Bayesian Networks
- Approximate Inference in Bayesian Networks

Constructing a Bayesian Network

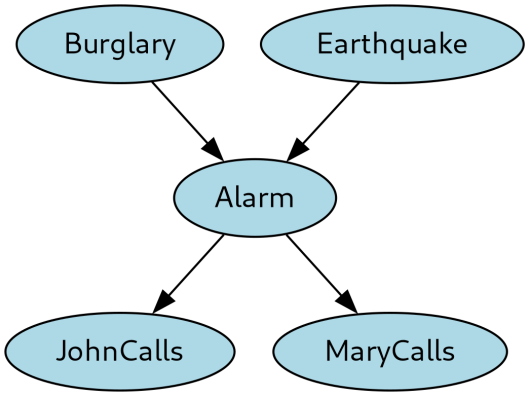
1. **Gather domain knowledge**
   - Identify key variables and their potential interactions
   - List all relevant random variables necessary to describe the system
2. **Order the nodes** according to cause-effects dependencies
   - So that the Bayesian network is minimal
3. For each node, **pick the minimum set of parents** $Parents(X_i)$
   - Add edges to represent the dependencies
   - Avoid redundant connections
4. **Estimate the conditional probability** $\Pr(X_i|Parents(X_i))$ for each node
   - Gather data or expert opinion
   - Use statistical techniques if necessary
5. **Validate the model**
   - Have domain experts review it
   - Ensure that the network is a Directed Acyclic Graph (DAG)
   - Test the network by predicting known outcomes and comparing with actual data

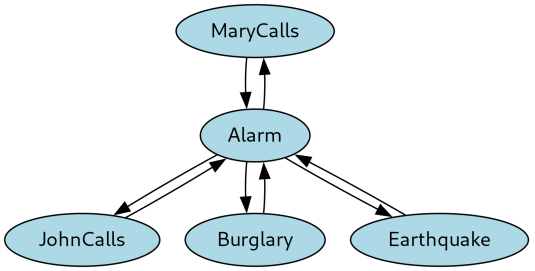
Bayesian Networks: Properties

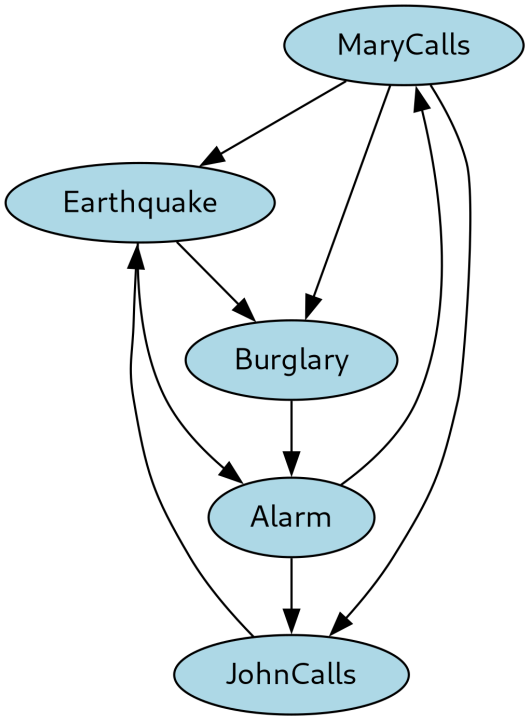
- Bayesian networks are a representation with several interesting properties
  - **Complete**
    - Encode all information in a joint probability
  - **Consistent** (non-redundant)
    - In a Bayesian network, there are no redundant probability values
    - One (e.g., a domain expert) can't create a Bayesian network violating probability axioms
  - **Compact** (locally structured, sparse)
    - Each subcomponent interacts directly with a limited number of other components
    - Typically yields linear (not exponential) growth in complexity
    - Sometimes we ignore real-world dependency to keep the graph simple
- In **fully connected systems**
  - Each variable is influenced by all others
  - The Bayesian network has the same complexity as the joint probability

Ordering of Nodes

- The complexity of the Bayesian network depends on the choice in ordering the nodes



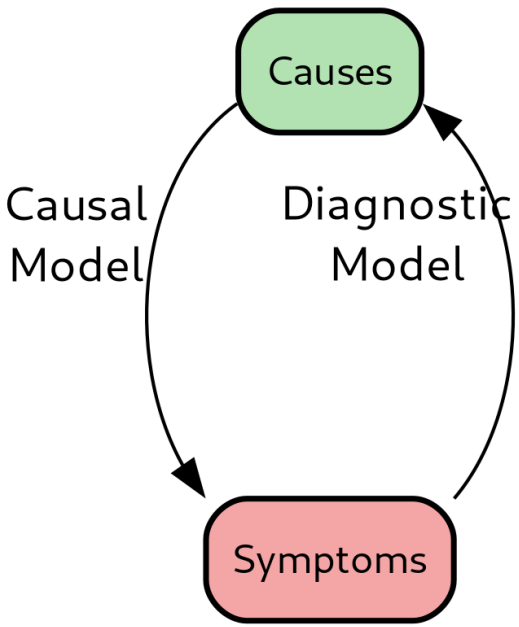




- The graph is "minimal" in terms of connectivity when all edges are causal

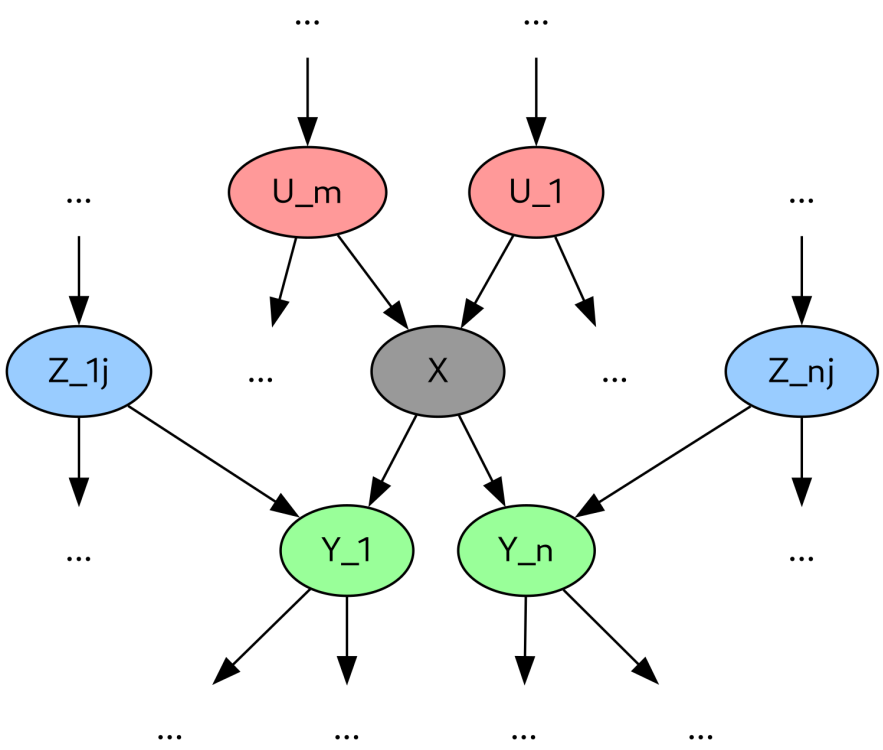
Causal vs Diagnostic Models

- A **causal model** goes from causes to symptoms
  - E.g., *Burglary* $\rightarrow$ *Alarm*
  - Simpler (i.e., fewer and more robust dependencies)
  - "Easier" to estimate
- A **diagnostic model** goes from symptoms to causes
  - E.g., *MaryCalls* $\rightarrow$ *Alarm* or *Alarm* $\rightarrow$ *Burglary*
  - Tenuous / unstable
  - Difficult to estimate
  - This is what we care about: use Bayes' rule to invert the probability



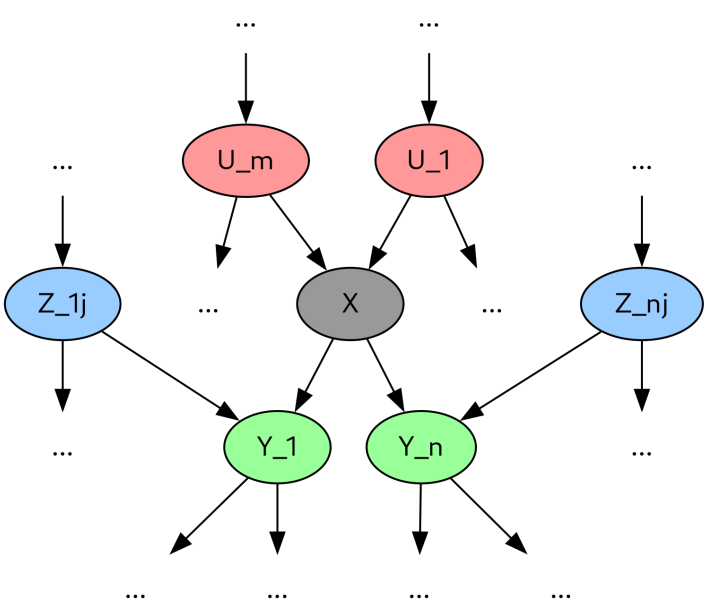
Markov Blanket of a Node

- The **Markov blanket** of a **node** X consists of:
  1. The **parents** of X
     - The nodes that influence X
  2. The **children** of X
     - The nodes that are directly influenced by X
  3. The **spouses** of X
     - The nodes that are parents of the children nodes
     - I.e., "co-parent"



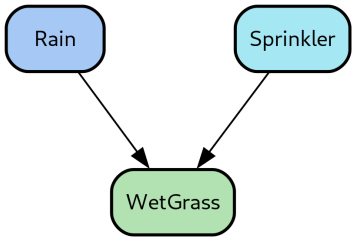
Conditional Independence on Markov Blanket

- In a Bayesian network, each variable is conditionally independent of:
  - **Its predecessors given its parents**
  - **All other nodes given its Markov blanket**, i.e., its parents, its children, and its spouses
- The **Markov blanket** of a node X_i:
  - Contains all the nodes necessary to predict the state of the node X_i, making the network irrelevant
  - Enables efficient and localized inference



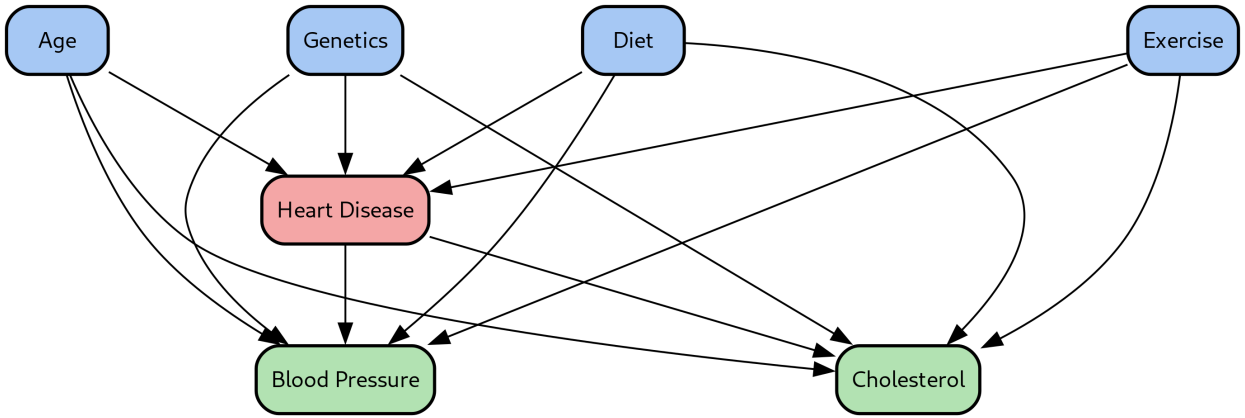
How Can a Node Be Influenced by Its Children?

- A **descendant can influence its ancestor** indirectly through *"explaining away"*
  - Evidence about the descendant can change what you believe about the ancestor through dependent paths
  - Information flows both ways in Bayesian networks
- E.g.,
  - Consider the Garden World
  - You know the grass is wet *WetGrass*
  - This evidence increases the probability of either causes *Rain* or *Sprinkler*
  - If you find out that the *Sprinkler* was on, this "explains away" the *WetGrass*, and the probability of *Rain* goes down
  - The evidence from a descendant *WetGrass* can update your belief about an ancestor *Rain*



Markov Blanket: Medical Example

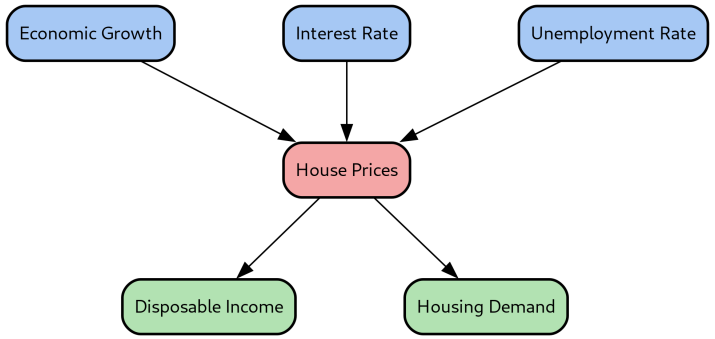
- Consider risk factors and outcomes for heart disease



- **Target node**
  - Heart disease
- **Parent nodes**
  - Risk factors of heart disease
  - Direct influence of H
- **Children nodes**
  - Outcomes of heart disease
  - Directly influenced by $H \rightarrow$
- **Spouse nodes**
  - A, G, D, E also influence BP and C
- Knowing the state of A, G, D, E, BP, C (Markov Blanket) allows to compute H, without any other information

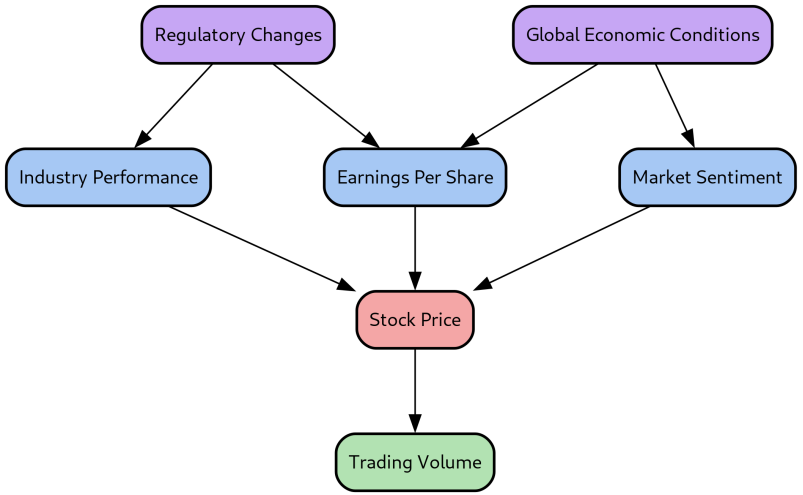
Markov Blanket: Economic Example

- Consider factors affecting house prices in a particular region
- **Target node**
  - House prices
- **Parent nodes**
  - Economic growth
  - Interest rate
  - Unemployment rate



- **Children nodes**
  - Disposable income
    - The house price affects how much money people have left after housing costs
  - Demand for houses
    - Higher prices can reduce demand

Markov Blanket: Finance Example



- Consider factors affecting an individual company's stock price
- **Target node**
  - *SP*: Stock Price
- **Parent nodes**
  - *IP*: Industry performance
  - *EPS*: Earnings per share
  - *MS*: Market sentiment
- **Children nodes**
  - *TV*: Trading volume
    - Changes in stock price influence how much stock is being traded
- **Grandparents nodes**
  - *RC*: Regulatory changes in the technology sector
    - Influences *IP* and *EPS*, but not directly *TV*
  - *GE*: Global economic conditions
    - Influences *MS* and *EPS*, but not directly *TV*
- You don't need to know *RC* and *GE* to estimate Stock Price

Specifying a Conditional Probability Table

- Conditional Probability Table (CPT) for a node requires $O(2^k)$ values in the worst case
  - Difficult to specify even with a small number of parents k
- Often, the relationship is not completely arbitrary, e.g.,
  - Deterministic nodes
  - Noisy logical relationships
  - Context-specific independence

Deterministic Nodes

- **Deterministic nodes** have values specified by their parents, without uncertainty, e.g.,
  - A logical relationship:
    - Useful when a condition is met if any of the sub-conditions are true
    - $IsNorthAmerican = IsCanadian \vee IsUS \vee IsMexican$
  - A numerical relationship:
    - $BestPrice = \min(Price_i)$
- **Note**
  - Deterministic nodes do not involve randomness or probability
  - Often used in models to simplify relationships and computations

Noisy Logical Relationships

- **Noisy logical relationships** (e.g., noisy-OR, noisy-MAX):
  - Are a probabilistic version of a logical relationship
  - Can be simpler to describe given k parents
- **Example**
- In propositional logic

$$Fever \iff Cold \vee Flu \vee Malaria$$

- In Bayesian networks
  - The assumptions are:
    1. All the possible causes are listed (you can use a leak node for "misc causes")
    2. There is uncertainty about the parents to cause the child node
    3. The probabilities of parents are independent
  - Under these assumptions:

$$\begin{aligned}&\Pr(Fever|parents(Fever))\\&\quad = 1 - \Pr(\neg Fever|Cold, \neg Flu, \neg Malaria)\cdot\\&\quad \Pr(\neg Fever|\neg Cold, \neg Flu, \neg Malaria)\cdot\end{aligned}$$

Context-specific Independence

- A variable exhibits **context-specific independence** if it is conditionally independent of its parents given certain values of others
- **Example**
  - *Damage* occurs depending on the *Ruggedness* of your car and whether an *Accident* occurred in that period:

$$\Pr(Damage \mid Ruggedness, Accident) = \begin{cases} d_1 & \text{if } Accident = True \\ d_2(Ruggedness) & \text{if } Accident = False \end{cases}$$

where $d1$ and $d2$ are distributions

Bayesian Networks with Continuous Variables

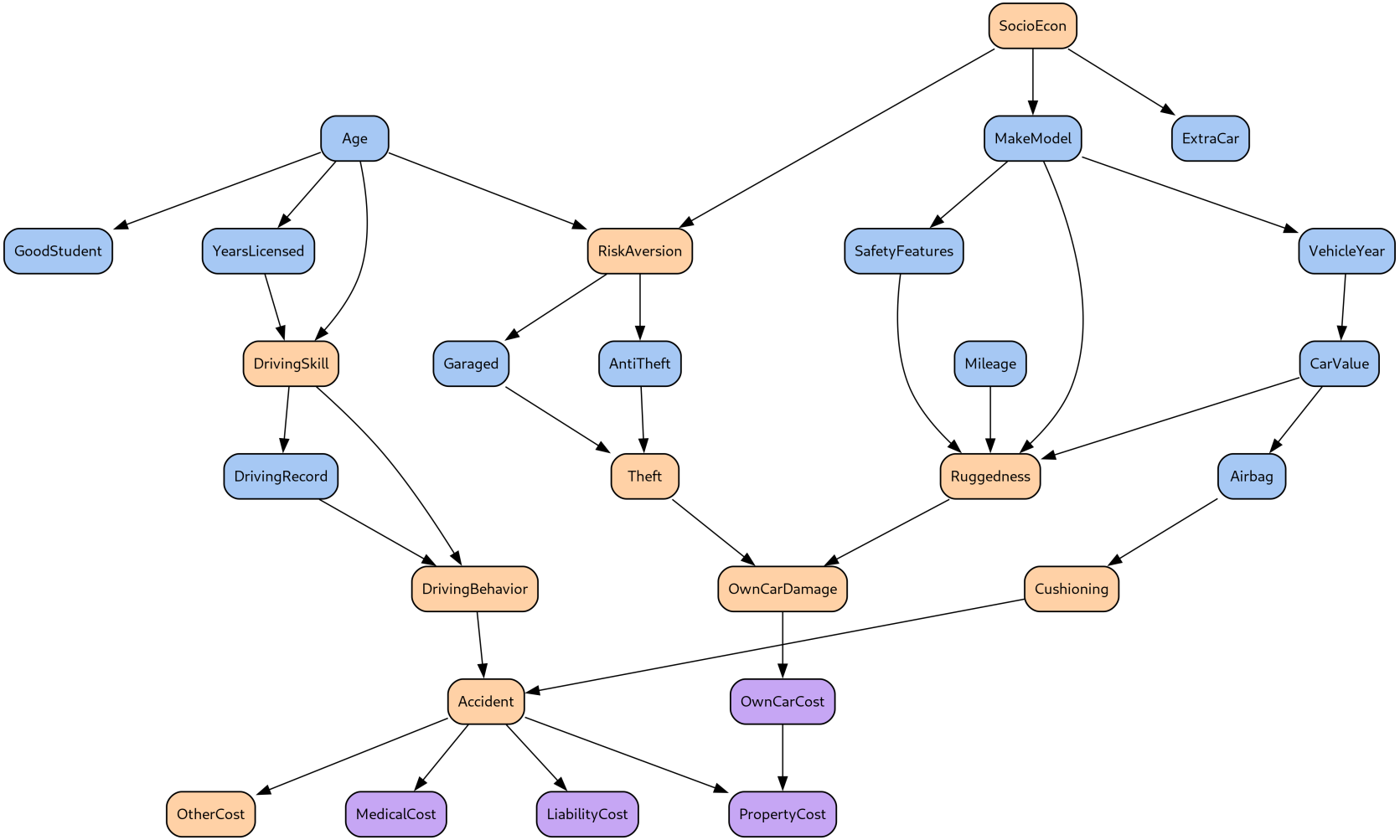
- Many real world problems involve continuous quantities
  - E.g., height, mass, temperature, money
- **Problem**: Conditional Probability Table (CPT) is not suitable for continuous RVs
- **Solution**:
  1. Discretization (i.e., use intervals)
     - Cons: loss of accuracy and large CPTs
  2. Continuous variables
     - Families of probability density functions (e.g., Gaussian distribution)
     - Non-parametric PDFs
- **Hybrid Bayesian networks** mix discrete and continuous variables, e.g.,
  - A customer buys a number of apples (discrete) depending on its cost (continuous)
  - Decide annual premium to charge to insure a vehicle based on applicant information (e.g., make model)

Bayesian Network: Car Insurance Company (1/2)

- A **car insurance company**:
  - Receive an application from an individual to insure a specific vehicle
  - Analyze information about the individual and its car
  - Decide on appropriate annual premium to charge
  - Pay out a claim, based on the type of claim
- Build a Bayesian network that captures the causal structure of the domain
  - **Input information**:
    - About the applicant: *Age*, *YearsWithLicense*, *DrivingRecord*, *GoodStudent*
    - About the vehicle: *MakeModel*, *VehicleYear*, *Airbag*, *SafetyFeatures*
    - About the driving situation: *Mileage*, *HasGarage*
  - Some input informations are **important but not available**:
    - *RiskAversion*
    - *DrivingBehavior*
  - **Type of claims**:
    - *MedicalCost*: injuries sustained by the applicant
    - *LiabilityCost*: lawsuits filed by other parties against applicant

Bayesian Network: Car Insurance Company (2/2)

- **Blue nodes**: information provided by the applicants
- **Brown nodes**: hidden variables (not observable)
- **Violet nodes**: target variables



- Semantics of Bayesian Networks
- Constructing a Bayesian Network
- ***Exact Inference in Bayesian Networks***
- Approximate Inference in Bayesian Networks

Exact Inference in Bayesian Networks

- **Goal of exact inference**
  - Compute the posterior $P(X|\underline{\boldsymbol{E}} = \underline{\boldsymbol{e}})$ for query variable X given evidence $\underline{\boldsymbol{e}}$
- **Variables involved**
  - Query variable X
  - Evidence variables $\underline{\boldsymbol{E}} = \{E_1, \ldots, E_m\}$
  - Hidden variables $\underline{\boldsymbol{Y}} = \{Y_1, \ldots, Y_\ell\}$
- **Inference by Enumeration**
  - Use full joint distribution and sum over all hidden variables:

$$P(X|e) = \alpha \sum_{Y} P(X, e, Y)$$

- **Variable Elimination**
  - Improves efficiency by caching intermediate results
  - Eliminates variables systematically to avoid redundant sums
  - Removing irrelevant variables
    - Variables not ancestors of query or evidence can be ignored
- **Problems**
  - Exact inference is efficient $O(n)$ for trees, but intractable $O(2^n)$ in general

Exact Inference in Bayesian Networks: Example

- You get a call from both John and Mary, what is the probability of the burglary?

$$P(Burglary|JohnCalls = True, MaryCalls = True)$$

- A conditional probability can be computed summing terms from the full joint distribution

$$\Pr(X|\underline{\boldsymbol{e}}) = \alpha \Pr(X, \underline{\boldsymbol{e}}) = \alpha \sum_{y} \Pr(X, \underline{\boldsymbol{e}}, \underline{\boldsymbol{y}})$$

- Terms of the joint distribution can be written as products of conditional probabilities from the Bayesian network

$$\Pr(b|j, m) = \alpha \Pr(B, j, m) = \alpha \sum_{e} \sum_{a} \Pr(B, j, m, e, a)$$

- Then the joint probability is written in terms of CPTs of the Bayesian network

$$\Pr(b|j, m) = \alpha \sum_{e} \sum_{a} \Pr(b) \Pr(e) \Pr(a|b, e) \Pr(j|a) \Pr(m|a)$$

- Semantics of Bayesian Networks
- Constructing a Bayesian Network
- Exact Inference in Bayesian Networks
- ***Approximate Inference in Bayesian Networks***

Monte Carlo Algorithms

- **Monte Carlo algorithms** are randomized sampling algorithms used to estimate quantities that are difficult to calculate exactly
  - E.g., samples from the posterior probability of a Bayes network
- **Pros**
  - The accuracy of the approximation depends on the number of samples generated
  - You can get arbitrarily close to the true probability distribution with enough samples
  - Is used in many branches of science
- **Cons**
  - Difficult to understand how the variables interact
  - Computationally intensive

Sampling from Arbitrary Distributions

- **Goal**: Sample from a discrete or continuous probability distribution
- **Solution**
  - Start with uniform random numbers $r \in [0, 1]$
  - Construct CDF (cumulative distribution function) $F(x)$
    - $F(x) = \Pr(X \leq x)$
  - For discrete distributions:
    - Create table of outcomes and cumulative probabilities
    - Find smallest outcome where $F(x) > r$
  - For continuous distributions:
    - Use inverse transform: $x = F^{-1}(r)$, e.g.,

$$F(x) = 1 - e^{-\lambda x} \rightarrow x = F^{-1}(r) = -\frac{1}{\lambda} \ln(1 - r)$$

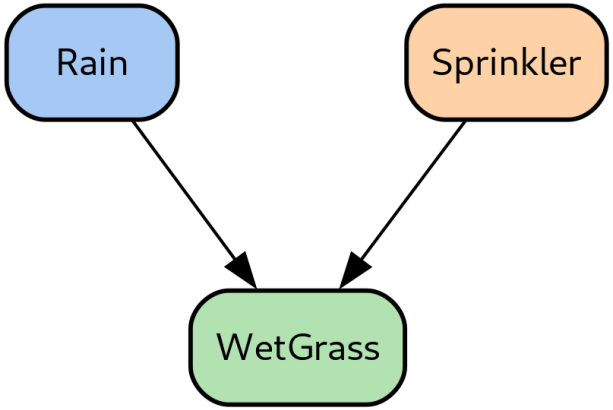
    - If F^{-1} has no closed form, use numerical methods

Sampling Bayesian Network Without Evidence

- **Goal**: Generate events from a Bayesian network without evidence (prior sampling)

- **Solution**
  - Sample variables in topological order (to ensure parents have values)
  - Source nodes have known unconditional probability distribution
    - E.g., $\Pr(Rain) = 0.5$
  - Conditional variable's probability distribution depends on parent's values
    - E.g., $\Pr(WetGrass|Rain = T) = 0.1$
  - Implement Bayesian network semantics, representing joint probability:

$$f_{PS}(x_1, ..., x_n) = \prod_{i=1}^{n} \Pr(x_i|parents(X_i))$$



Consistency of Sampling

- **Consistency of estimation**: distribution from prior sampling converges to true probability as $N \to \infty$
- If N_{PS} is the number of times event $x_1, ..., x_n$ occurs:

$$\lim_{N \to \infty} \frac{N_{PS}(x_1, ..., x_n)}{N} = \Pr(x_1, ..., x_n)$$

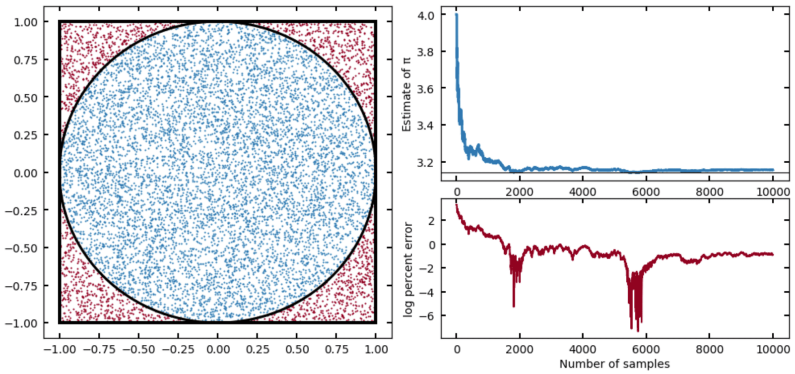
- Estimate probability using:

$$\Pr(x_1, ..., x_n) \approx \frac{N_{PS}(x_1, ..., x_n)}{N}$$

  - Converges with rate $\frac{1}{\sqrt{N}}$

Rejection Sampling

- **Rejection sampling** is a method for sampling from a hard-to-sample distribution
- **Goal**: Compute $\Pr(X = x | E = e)$ when evidence e is rare
  1. Generate samples from the prior distribution
     - Estimate $\Pr(x, e)$
  2. Reject samples not matching evidence, i.e., $X \wedge E \neq e$
     - Remaining samples $X \wedge E = e$ estimate $\Pr(X, E = e)$
  3. Count occurrences of $X = x$ in remaining samples $X \wedge E = e$
     - Estimate $\Pr(X = x | E = e)$



- **Example**:

Rejection Sampling: Pros and Cons

- **Pros**
  - Consistent estimate
    - Converges to true value as number of samples increases
- **Cons**
  - Many samples are rejected, depending on rarity of $\Pr(E = e)$
  - Fraction of samples matching evidence e decreases exponentially with more evidence variables
    - Curse of dimensionality
    - Not suitable for complex systems
  - Difficult with continuous variables
    - E.g., $\Pr(E = e)$ is theoretically 0 due to limited floating-point precision

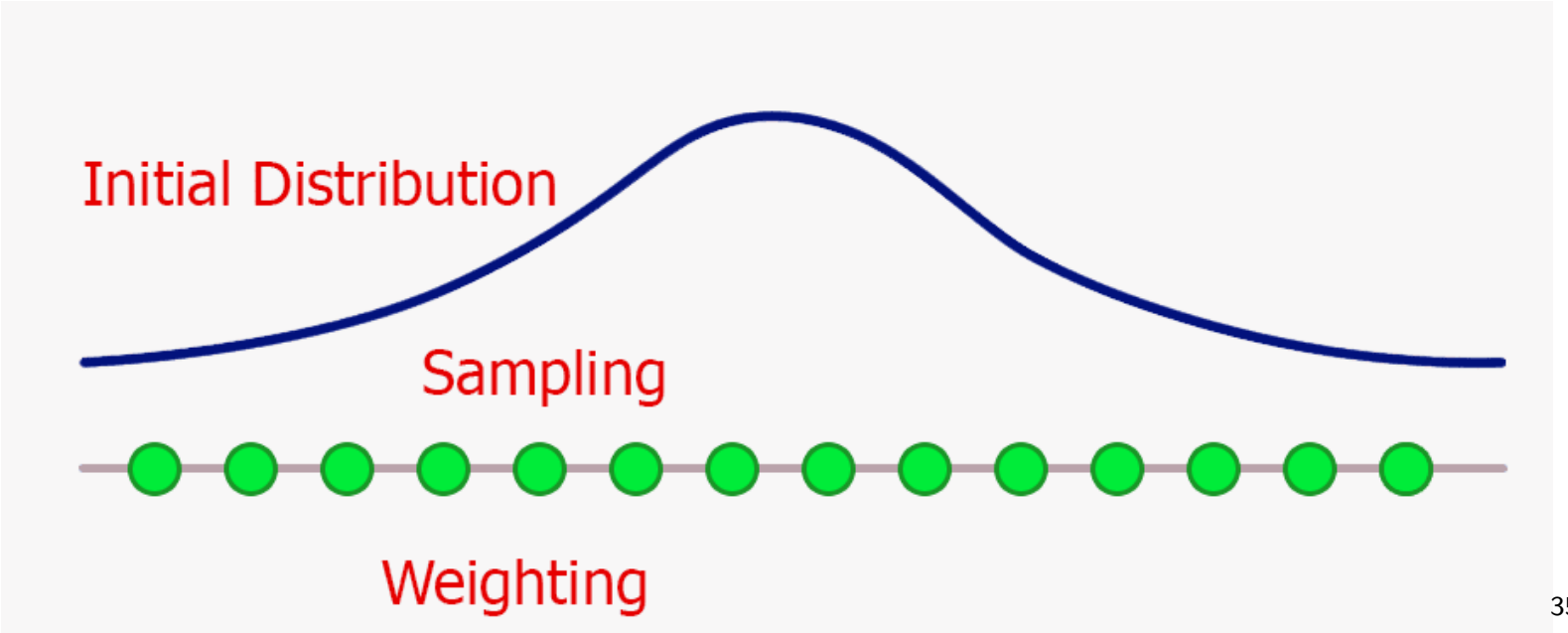
Importance Sampling

::: columns :::: {.column width=60%}

- **Importance sampling**
  - Draw samples from "easier" distribution $Q(X)$
  - Weight each sample by importance weight $w = \frac{\Pr(X)}{Q(X)}$
  - Estimate probability by averaging weighted samples:

$$E[f(X)] \approx \frac{1}{N} \sum_{i=1}^{N} w_i f(X_i)$$

:::: :::: {.column width=35%}



Markov Chain Monte Carlo

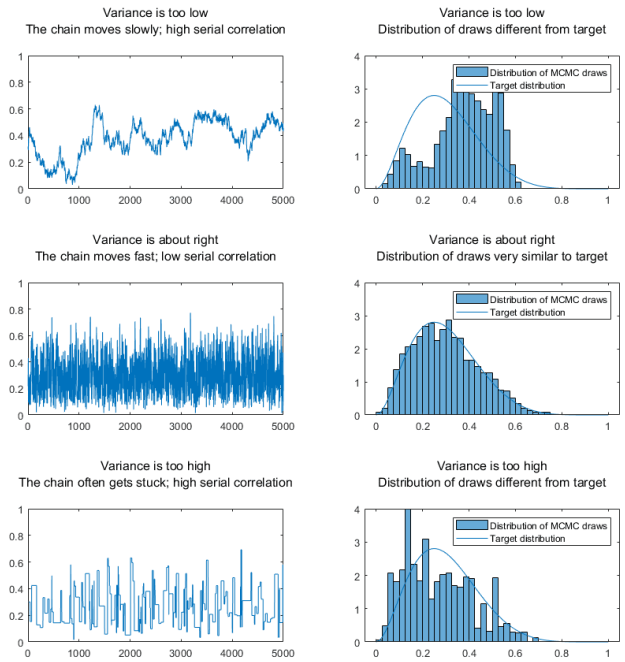
- This is one of the top 10 most mind blowing algorithms in history
  - Euclide's GCD
  - Fundamental theorem of calculus
  - Quicksort
  - Fast Fourier Transform
  - Viterbi algorithm
  - MCMC sampling
  - Kalman filter
  - RSA Algorithm
  - ...
- Invented by Ulam, Von Neumann, Metropolis and others during the Manhattan Project (1940)
  - Used to solve high-dimensional integrals, Bayesian inference, ...
- **Purpose**: Approximate inference for Bayesian networks when exact inference is hard
  - MCMC differs from rejection and importance sampling
  - Make random changes to preceding sample instead of generating each sample independently
  - **Magic**: two very different objects (Markov Chains) and Bayesian Networks are connected

Markov Chain Construction

- A **Markov chain** is a "random walk" through states, where the future depends only on the present
  - *Sequence of states*: $\underline{\boldsymbol{x}}^{(0)}, \underline{\boldsymbol{x}}^{(1)}, \underline{\boldsymbol{x}}^{(2)}, \ldots$
  - *Initial state*: starting configuration
  - *Transition probabilities*: $\Pr(\underline{\boldsymbol{x}} \to \underline{\boldsymbol{x}}')$
  - After t steps, distribution is $\pi_t(\underline{\boldsymbol{x}})$
  - When $\pi_t(\underline{\boldsymbol{x}}) = \pi_{t+1}(\underline{\boldsymbol{x}})$, chain reaches stationary distribution
- **Transition operator**: moves from one state to another:
  - Gibbs sampling: resample one variable given its Markov blanket
  - Metropolis–Hastings: propose a new state, then accept/reject based on a probability ratio
- There are algorithms generate a Markov Chain from a Bayesian network
- Under certain conditions:
  - Ergodicity: chain can reach any state
  - Aperiodicity: chain does not get stuck in cycles stationary distribution equals the **posterior distribution** over non-evidence variables given evidence

Markov Chain Monte Carlo: Mixing

- **Mixing** describes how quickly a Markov chain forgets its starting point and explores the whole state space efficiently
  - A well-mixed chain:
    - Moves between different high-probability regions often
    - Has low correlation between successive samples
  - Poor mixing:
    - Chain gets stuck in one mode for a long time
    - Leads to biased estimates and high variance
- In practice:
  - Discard initial samples as a burn-in period (before convergence)
  - After convergence, collected samples approximate the true posterior
- **Example**
  - If sampling from a bimodal



Gibbs Sampling in Bayesian Networks

- Special case of Markov Chain Monte Carlo (MCMC) method that samples one variable at a time
- **Algorithm**:
  - Start with an initial complete assignment to all non-evidence variables
  - Keep evidence variables fixed at observed values
  - For each non-evidence variable X_i:
    - Sample X_i from $P(X_i|\text{MB}(X_i))$ where $\text{MB}(X_i)$ is the Markov blanket, i.e., parents, children, spouse of a node
- **Example**:
  - Weather network: $P(Cloudy|Sprinkler, Rain, WetGrass)$
  - Fix $WetGrass = true$, $Sprinkler = true$
  - Sample *Cloudy* and *Rain* iteratively
- **Pros**
  - Simple to implement for any Bayesian network
  - Handles large, complex graphs with local updates
- **Cons**

Metropolis–Hastings Sampling

- More general Markov Chain Monte Carlo method than Gibbs sampling
- **Algorithm**
  - Start at a current state $\underline{\boldsymbol{x}}$
  - Propose a new state $\underline{\boldsymbol{x}}'$ from a proposal distribution $q(x'|x)$, e.g.,
    - With 95% probability Gibbs sampling
    - Otherwise use importance sampling
  - Compute the acceptance probability:
    - $A(x, x') = \min(1, \frac{\pi(x')q(x|x')}{\pi(x)q(x'|x)})$
  - Move to x' with probability $A(x, x')$, otherwise stay at x
- **Intuition**
  - Propose local moves
  - Accept if they lead to higher probability, or sometimes accept lower-probability states to explore
  - Balances exploration and exploitation to avoid getting stuck in local modes
- **Pros**
  - Very flexible: works with any proposal distribution